

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 2– CASE STUDY ANALYSIS

Course Code:	CSA65	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO2 – Precision (BL2, BL3)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	10% of CIA	Total Marks:	2 * 50 = 100 (2 Questions)
CO2 Statement:	<i>Apply prompt engineering techniques and utilize pre-trained Large Language Models through modern AI frameworks and APIs to solve real-world problems (BL2, BL3)</i>		
Student Name:	Y. Jayesh Ranjan	Reg. No.:	192472201
Section / Batch:	CSA – AI	Date:	02/08/2026

Instructions to Students

- Answer **2 questions**. Each question carries **50 marks**. **Total: 100 Marks**.
- Demonstrate a **deep understanding of the concepts** relevant to the given case study.
- Identify and analyze the **key problems, requirements, challenges, and constraints** presented in the case.
- Apply appropriate **technical concepts, methods, and approaches** to propose and justify a suitable solution.
- Discuss the **real-world implications, applications, benefits, limitations, and challenges** of the proposed solution.
- Provide **thorough, evidence-based, and well-structured analysis**, supported by suitable examples, diagrams, architectures, or workflows wherever applicable.
- Present the analysis with a **clear and professional structure**, using appropriate headings and logical organization.
- Use **relevant references and proper citation practices** and maintain clear academic and technical writing.

Code of Conduct

I Y. Jayesh Ranjan – 192472201 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Y. Jayesh Ranjan

SECTION A – CASE STUDY ANALYSIS

26. Function Calling for University Services

A university assistant should check attendance, retrieve examination information, and generate student notifications.

Analyze which functions should be provided to the LLM and how the system should determine when a function needs to be called. Discuss suitable structured outputs.

1. Introduction

Modern universities increasingly use AI assistants to automate student services such as checking attendance, retrieving examination information, and sending notifications. These tasks require access to real-time university databases rather than relying solely on the LLM's internal knowledge. Function

calling enables the LLM to invoke external functions or APIs to fetch accurate information and perform actions.

2. Case Study Analysis

Problem Statement

The university assistant should:

- Check attendance
- Retrieve examination information
- Generate student notifications

The challenge is determining which function to call, when to call it, and how to return the results in a structured format.

3. Requirements

The AI assistant should:

- Understand user requests.
 - Call the correct university function.
 - Retrieve real-time data.
 - Return structured outputs.
 - Handle missing or invalid information gracefully.
 - Protect student privacy and security.
-

4. Key Problems

Problem 1

Selecting the correct function.

Example:

"Show my attendance."

The assistant should call the attendance function.

Problem 2

Authentication.

Student identity must be verified before accessing records.

Problem 3

Invalid requests.

Example:

Exam information requested for a non-existent course.

Problem 4

Unavailable services.

The database server may be temporarily unavailable.

5. Proposed Functions

Function 1: Check Attendance

Function Name

`checkAttendance(studentID)`

Returns:

- Attendance percentage
 - Total classes
 - Classes attended
-

Function 2: Retrieve Examination Information

Function Name

`getExamDetails(courseCode)`

Returns:

- Exam date
- Time
- Hall
- Subject

Function 3: Generate Student Notification

Function Name

`sendNotification(studentID,message)`

Returns:

- Notification status
 - Timestamp
-

6. When Should the LLM Call a Function?

The system should determine function calls based on user intent.

User Request	Function Called
"Check my attendance."	<code>checkAttendance()</code>
"When is my DBMS exam?"	<code>getExamDetails()</code>
"Send exam reminder."	<code>sendNotification()</code>

The workflow is:

1. User submits a request.
 2. LLM identifies the intent.
 3. Required parameters (Student ID, Course Code, etc.) are extracted.
 4. Appropriate function/API is invoked.
 5. The function returns real-time data.
 6. LLM formats and presents the response.
-

7. Suitable Structured Outputs

A structured JSON response makes integration easier.

Attendance Output

```
{  
  
  "student_id": "21CS001",  
  
  "attendance_percentage": 92,  
  
  "total_classes": 50,
```

```
"classes_attended": 46
```

```
}
```

Examination Details

```
{
```

```
"course": "DBMS",
```

```
"exam_date": "15 Dec 2026",
```

```
"time": "10:00 AM",
```

```
"hall": "Block A - Room 204"
```

```
}
```

Notification Output

```
{
```

```
"status": "Sent",
```

```
"message": "Exam starts tomorrow at 10:00 AM.",
```

```
"timestamp": "2026-12-14T09:00:00"
```

```
}
```

8. System Workflow

Student Request

|



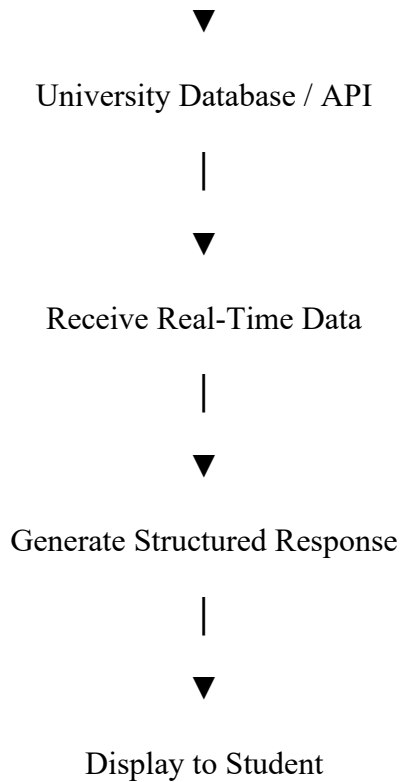
Intent Detection by LLM

|



Select Appropriate Function

|



9. Benefits

- Accurate real-time information
 - Faster student support
 - Reduced manual workload
 - Consistent responses
 - Easy integration with university systems
 - Improved student experience
-

10. Limitations

- Requires secure authentication.
 - Dependent on API availability.
 - Network failures may affect responses.
 - Privacy and data protection must be ensured.
-

11. Real-World Applications

- Student information systems
- University help desks
- Learning Management Systems (LMS)
- Exam management portals

- Mobile campus applications
-

12. Conclusion

Function calling enables an LLM to interact with external university systems for accurate and real-time services. By selecting the appropriate function based on user intent and returning structured outputs, the assistant can efficiently support attendance tracking, examination queries, and student notifications while ensuring consistency, security, and scalability.

13. References

1. OpenAI. *Function Calling Documentation*.
2. OpenAI. *Prompt Engineering Guide*.
3. Brown et al. (2020). *Language Models are Few-Shot Learners*.
4. Russell, S., & Norvig, P. *Artificial Intelligence: A Modern Approach*.

35. Structured Output for Enterprise Reports

An enterprise wants an LLM to generate reports containing fixed fields such as summary, key findings, risk level, recommendations, and priority.

Analyze how structured prompting can ensure consistent output. Discuss what should happen when information required for a field is unavailable.

1. Introduction

Large enterprises use Large Language Models (LLMs) to automatically generate business reports. These reports are often consumed by executives, managers, auditors, and analysts. Therefore, every report must maintain the same format, regardless of the input data.

The case study requires the LLM to generate reports containing fixed fields:

- Summary
- Key Findings
- Risk Level
- Recommendations
- Priority

The main challenge is ensuring consistency while also handling situations where information required for a field is unavailable.

2. Case Study Analysis

Problem Statement

The enterprise wants every report to follow an identical structure.

Without structured prompting:

- Reports may contain different headings.
- Some fields may be omitted.
- Information may appear in random order.
- Different wording may confuse decision makers.

Therefore, a structured output mechanism is required.

3. Requirements

The system should:

- Generate reports using fixed fields.
 - Maintain the same output format.
 - Handle incomplete data safely.
 - Prevent hallucination.
 - Produce machine-readable outputs if required.
 - Support easy integration with enterprise software.
-

4. Challenges

Challenge 1

Incomplete input data.

Example:

Risk information may not be available.

Challenge 2

Different report styles.

Different prompts may produce inconsistent reports.

Challenge 3

Hallucination

LLMs may invent information when data is missing.

Challenge 4

Integration

Enterprise software requires standardized outputs.

5. Constraints

- Reports must always contain every required field.
 - Missing information must never be fabricated.
 - Reports should be easy to parse.
 - Output should remain professional.
-

6. Proposed Solution

The enterprise should use **Structured Prompting** together with a predefined report template.

Example prompt:

Generate an enterprise report using the following structure.

Summary:

Key Findings:

Risk Level:

Recommendations:

Priority:

Rules:

- Never change field names.
- If information is unavailable, write "Information Not Available".
- Do not invent missing facts.
- Maintain professional language.

This prompt forces consistent output.

7. Handling Missing Information

If a field has no available information:

Instead of generating false information, the LLM should return:

Information Not Available

or

Not Provided

Example:

Field	Output
Summary	Sales increased by 15%
Key Findings	New customers increased
Risk Level	Information Not Available
Recommendations	Expand production
Priority	High

This prevents hallucination and increases reliability.

8. Technical Concepts Used

Structured Prompting

Provides predefined sections.

Output Schema

Every report follows the same schema.

Validation

Checks whether every required field exists.

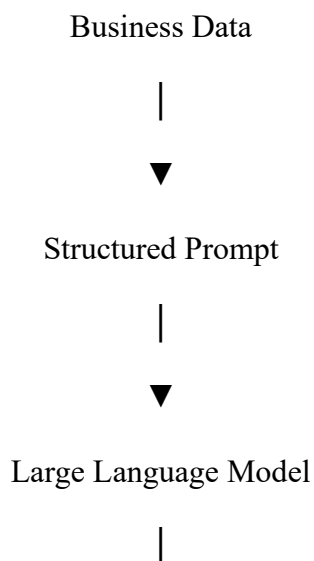
Missing Value Handling

Uses placeholders instead of fabricated information.

JSON Output (Optional)

```
{  
  "summary": "Sales increased by 15%",  
  "key_findings": "Customer growth observed",  
  "risk_level": "Information Not Available",  
  "recommendations": "Increase inventory",  
  "priority": "High"  
}
```

9. Workflow



▼
Output Validation

|
▼
Final Enterprise Report

10. Example

Input

Quarterly sales increased.

Customer satisfaction improved.

Risk data unavailable.

Output

Summary:

Quarterly sales improved significantly.

Key Findings:

Customer satisfaction increased.

Risk Level:

Information Not Available.

Recommendations:

Increase production capacity.

Priority:

High.

11. Benefits

- Consistent reports
- Easy automation
- Better decision making
- Prevents hallucination
- Easier software integration
- Professional documentation

12. Limitations

- Depends on prompt quality.
- Cannot infer unavailable information.
- Requires validation.
- Needs regular prompt updates.

13. Real-World Applications

- Financial reporting
- Healthcare reports
- Insurance claims
- Audit reports
- Cybersecurity reports
- Manufacturing reports

14. Conclusion

Structured prompting ensures that enterprise reports always follow the same format. Using predefined fields, validation rules, and placeholders for missing information improves consistency, reliability, and trustworthiness while reducing hallucinations and simplifying enterprise integration.

15. References

1. OpenAI. *Prompt Engineering Guide*.
2. Brown et al. (2020). *Language Models are Few-Shot Learners*.
3. Microsoft Azure OpenAI Documentation.
4. Google Prompt Design Guide.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Depth	30%	Deep, accurate understanding of pre-training/fine-tuning/RLHF concepts	Good understanding with minor gaps	Superficial understanding of concepts	Misunderstands core concepts
Real-World Implications	25%	Draws well-reasoned real-world implications and comparisons	Reasonable implications drawn	Weak or generic implications	No meaningful analysis presented
Analytical Rigor	20%	Analysis is thorough, evidence-based, and well-structured	Mostly thorough analysis	Partial analysis with gaps in evidence	Superficial, unstructured analysis
Report Structure	15%	Report/slides professionally structured, easy to follow	Clear structure with minor issues	Structure unclear in places	Poorly structured submission
Citation & Academic Writing	10%	Properly cites sources; clear academic writing	Mostly cited, minor lapses	Inconsistent citation practice	No citation or referencing

Marks Evaluation:

Question No.	Conceptual Depth(30%)	Real-World Implications (25%)	Analytical Rigor (20%)	Report Structure (15%)	Citation & Academic Writing(10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____